

SOFE 4630U: Cloud Computing

Phase 1: Monolithic Implementation for a Cloud-based Scenario Extraction and Processing from the NGSIM (US-101) Dataset

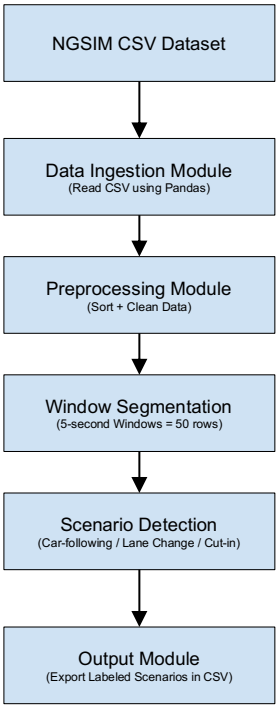
Group 12:
Anuj Shukla (100895219)
Ishmeet Kaur (100809450)
Brandon Ho (100754357)
Gabriele Martorano (101046703)

VIDEO LINK: https://youtu.be/zjIOVaH3rUU

Link to the specific raw dataset we used: trajectories-0820am-0835am.csv

1. System Architecture

The following architecture diagram demonstrates the program flow of the monolithic system, at a high level. It is intended to help the user and developers track the data as it moves through the pipeline.



2. Data Flow

The system processes vehicle trajectory data from the NGSIM US-101 dataset to detect common highway driving scenarios such as car-following, lane changes, and cut-ins. The system is designed as a modular data pipeline that sequentially transforms raw trajectory data into extracted traffic scenarios. The pipeline is shown in section 1 with the architecture diagram as a visual representation.

Data Ingestion

The pipeline begins by loading the raw traffic trajectory dataset from a CSV file. The dataset contains detailed vehicle trajectory information including vehicle identifiers, frame numbers,

lane positions, velocities, and spatial coordinates. Each row in the dataset represents the state of a vehicle at a particular time frame.

Data Preprocessing

The preprocessing stage prepares the dataset for analysis by:

- Cleaning the dataset
- Ensuring numerical data types for trajectory attributes
- Sorting vehicle observations by frame order
- Preparing vehicle trajectories for time-based segmentation

This step ensures that the dataset can be processed efficiently during scenario detection.

Sliding Window Segmentation

To analyze vehicle behavior over time, the system applies a sliding window segmentation technique.

Each window represents approximately 5 seconds of traffic activity, which corresponds to 50 frames in the dataset (the dataset is sampled at approximately 10 frames per second).

Instead of grouping by individual vehicles, the system constructs frame-based windows, meaning that each window contains all vehicles present during that time interval. This allows the algorithm to capture interactions between vehicles, which are necessary to detect scenarios such as cut-ins and car-following behavior.

The window then becomes the unit of analysis for scenario detection.

Scenario Detection and Output

For each window, the system analyzes the behavior of the vehicles and classifies the window into one of several predefined traffic scenarios. The results are stored in a structured output file containing:

- Start frame of the window
- End frame of the window
- Number of vehicles in the window
- Detected scenario label

The final output is stored in a CSV file for further analysis or visualization.

3. Scenario Detection Logic

The system detects three types of driving scenarios:

1. Car Following
2. Lane Change
3. Cut-In

Each scenario is defined using conditions derived from vehicle trajectory attributes.

Lane Change Detection

A lane change is detected when a vehicle transitions from one lane to another during the observation window.

The detection algorithm checks whether the lane identifier (Lane_ID) for any vehicle changes during the window.

Detection rule:

If the number of unique Lane_ID values > 1

→ Lane Change detected

This indicates that a vehicle has moved laterally across lane boundaries within the observation period.

Car-Following Detection

Car-following occurs when one vehicle travels closely behind another vehicle in the same lane while maintaining similar speed.

The algorithm checks:

- Whether two vehicles are in the same lane
- Whether the distance between them is less than 30 meters
- Whether the velocity difference is less than 5 m/s

Detection rule:

Same lane
AND distance < 30 m

AND velocity difference < 5 m/s
→ Car Following

This scenario represents typical highway driving behavior where vehicles maintain safe following distances.

Cut-In Detection

A cut-in occurs when a vehicle moves from another lane into the ego vehicle's lane and positions itself directly ahead of the ego vehicle.

The algorithm checks for vehicles that:

1. Start in a different lane
2. End in the ego vehicle's lane
3. Appear ahead of the ego vehicle within a short distance

Detection rule:

`Start lane ≠ Ego lane`

AND End lane = Ego lane

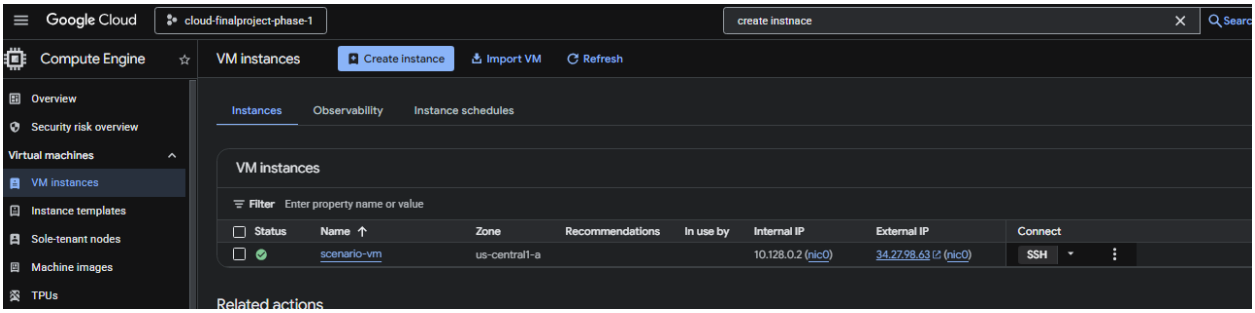
AND vehicle appears ahead within 30 m

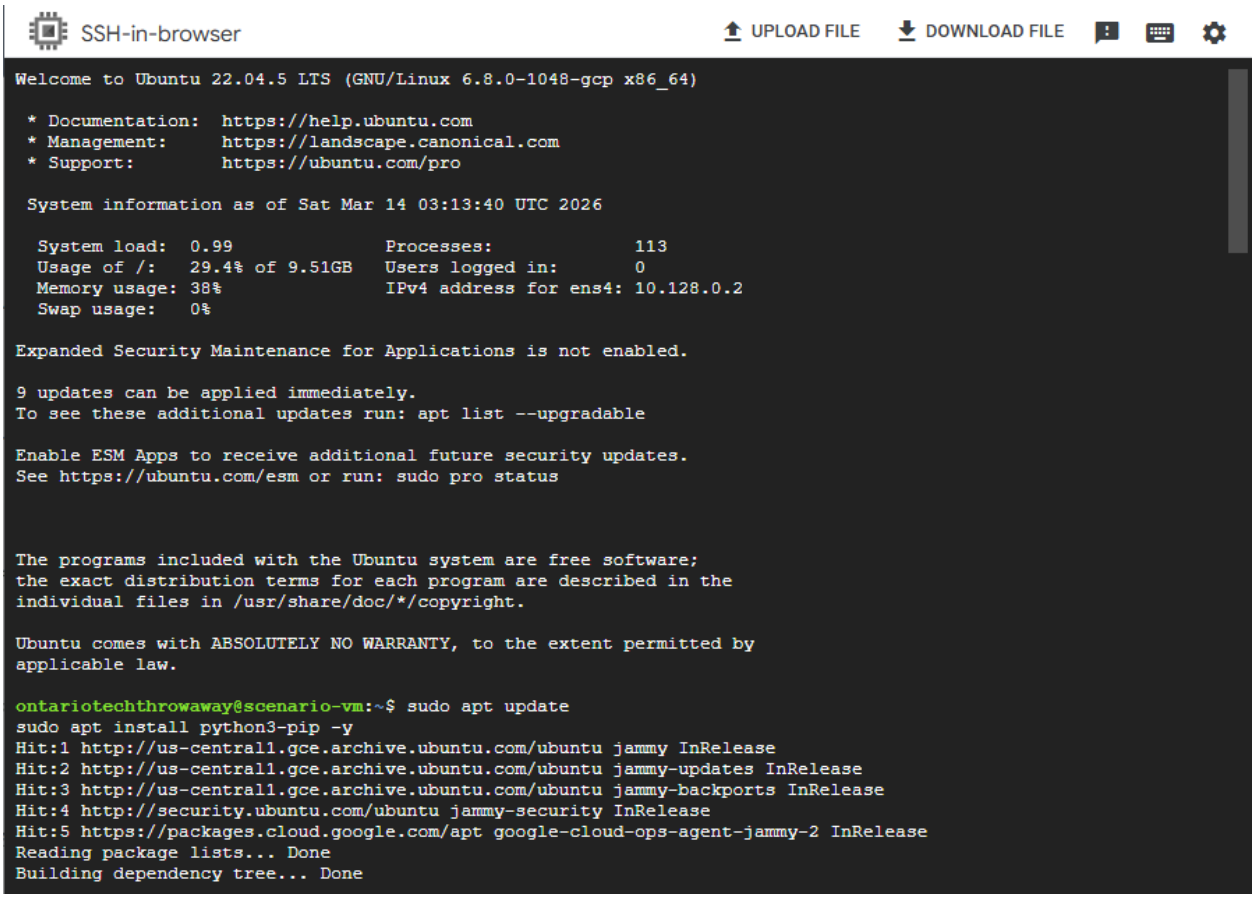
→ Cut-In

Cut-in events are important to detect because they represent potentially disruptive driving maneuvers that can affect traffic flow and safety.

4. Cloud Deployment

The following screenshots demonstrate the process of deploying the monolithic application to Google Cloud using a VM. As outlined in the proposal




SSH-in-browser
UPLOAD FILE
DOWNLOAD FILE
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1048-gcp x86_64)
* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/pro
System information as of Sat Mar 14 03:13:40 UTC 2026
System load: 0.99 Processes: 113
Usage of /: 29.4% of 9.51GB Users logged in: 0
Memory usage: 38% IPv4 address for ens4: 10.128.0.2
Swap usage: 0%
Expanded Security Maintenance for Applications is not enabled.
9 updates can be applied immediately.
To see these additional updates run: apt list --upgradable
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
ontariotechthrowaway@scenario-vm:~$ sudo apt update
sudo apt install python3-pip -y
Hit:1 http://us-central1.gce.archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://us-central1.gce.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:3 http://us-central1.gce.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:5 https://packages.cloud.google.com/apt google-cloud-ops-agent-jammy-2 InRelease
Reading package lists... Done
Building dependency tree... Done

```
Setting up dpkg-dev (1.21.1ubuntu2.6) ...
Setting up libjs-sphinxdoc (4.3.2-1) ...
Setting up libgcc-11-dev:amd64 (11.4.0-1ubuntu1~22.04.3) ...
Setting up gcc-11 (11.4.0-1ubuntu1~22.04.3) ...
Setting up cpp (4:11.2.0-1ubuntu1) ...
Setting up libc6-dev:amd64 (2.35-0ubuntu3.13) ...
Setting up libtiff5:amd64 (4.3.0-6ubuntu0.12) ...
Setting up libfontconfig1:amd64 (2.13.1-4.2ubuntu5) ...
Setting up gcc (4:11.2.0-1ubuntu1) ...
Setting up libexpat1-dev:amd64 (2.4.7-1ubuntu0.7) ...
Setting up libgd3:amd64 (2.3.0-2ubuntu2.3) ...
Setting up libstdc++-11-dev:amd64 (11.4.0-1ubuntu1~22.04.3) ...
Setting up zlib1g-dev:amd64 (1:1.2.11.dfsg-2ubuntu9.2) ...
Setting up libc-devtools (2.35-0ubuntu3.13) ...
Setting up g++-11 (11.4.0-1ubuntu1~22.04.3) ...
Setting up libpython3.10-dev:amd64 (3.10.12-1~22.04.15) ...
Setting up python3.10-dev (3.10.12-1~22.04.15) ...
Setting up g++ (4:11.2.0-1ubuntu1) ...
update-alternatives: using /usr/bin/g++ to provide /usr/bin/c++ (c++) in auto mode
Setting up build-essential (12.9ubuntu3) ...
Setting up libpython3-dev:amd64 (3.10.6-1~22.04.1) ...
Setting up python3-dev (3.10.6-1~22.04.1) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for libc-bin (2.35-0ubuntu3.13) ...
Scanning processes...
Scanning linux images...

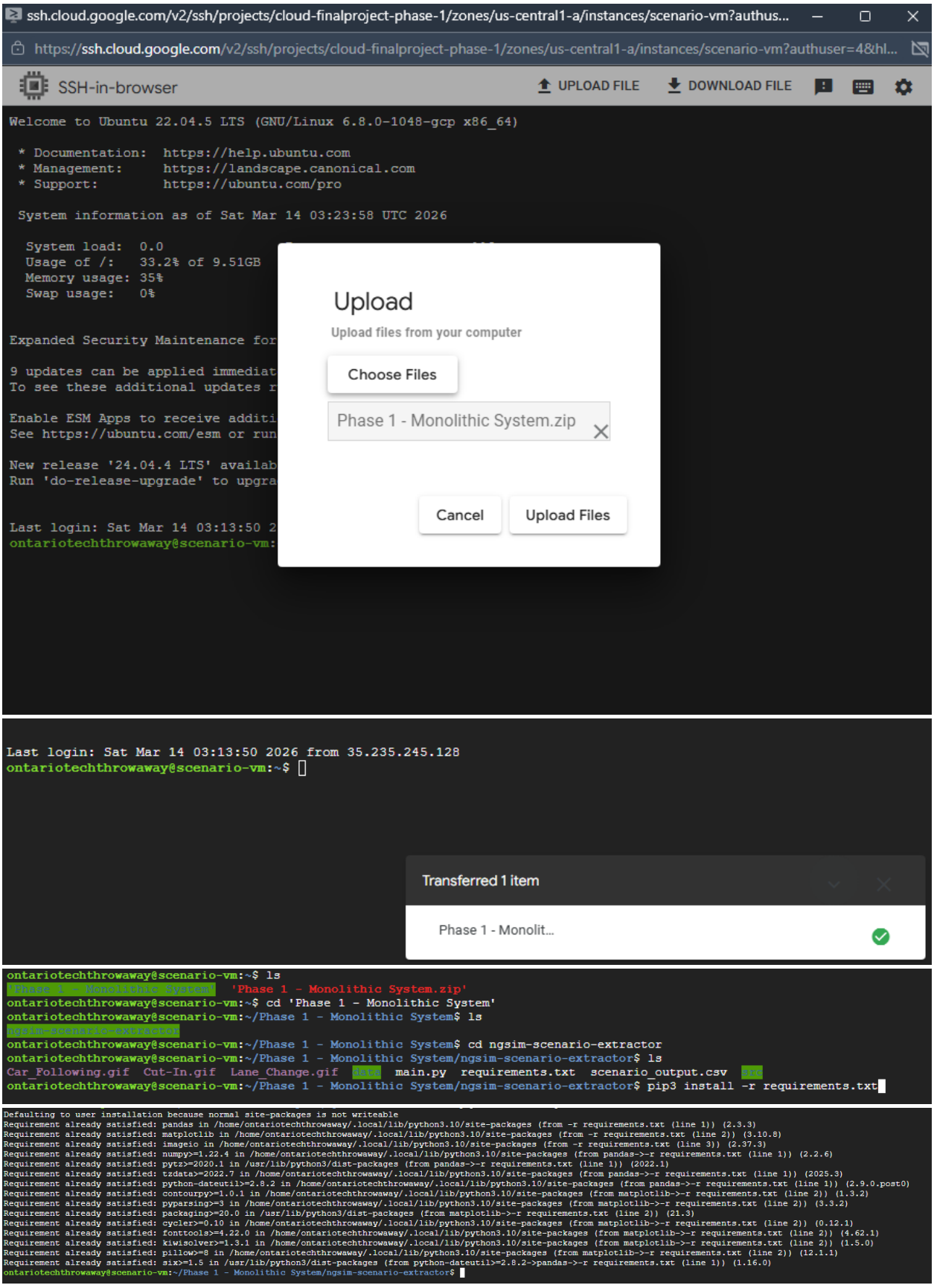
Running kernel seems to be up-to-date.

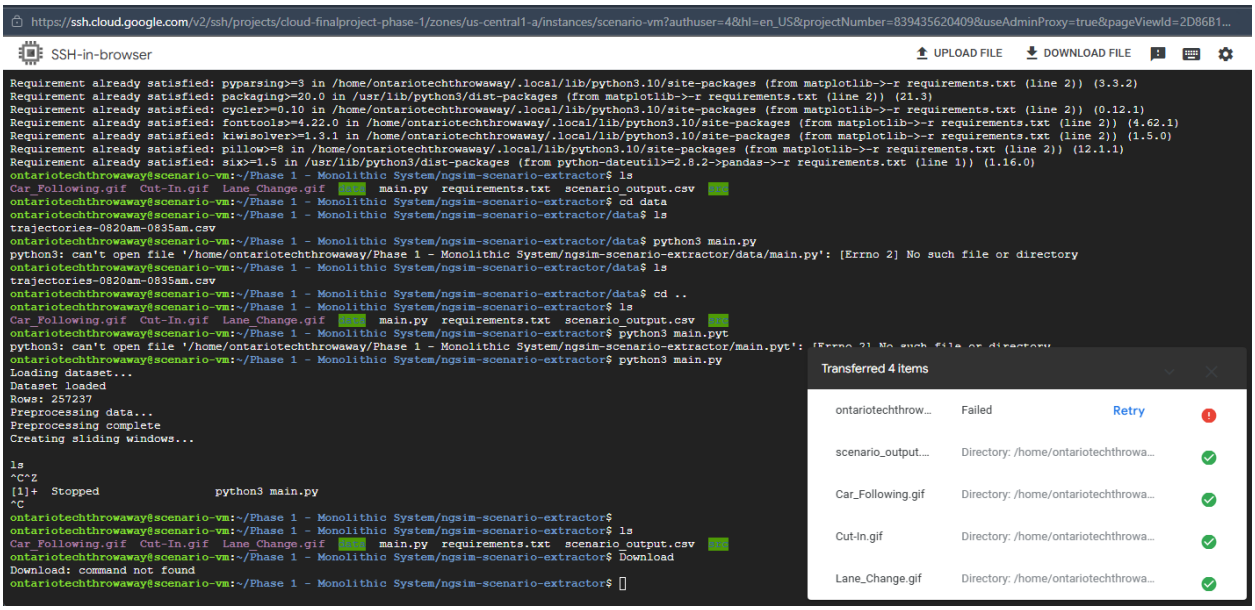
No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ontariotechthrowaway@scenario-vm:~$ python3 --version
Python 3.10.12
ontariotechthrowaway@scenario-vm:~$
```


ssh.cloud.google.com/v2/ssh/projects/cloud-finalproject-phase-1/zones/us-central1-a/instances/scenario-vm?authus...
https://ssh.cloud.google.com/v2/ssh/projects/cloud-finalproject-phase-1/zones/us-central1-a/instances/scenario-vm?authuser=4&hl...
SSH-in-browser
UPLOAD FILE
DOWNLOAD FILE
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1048-gcp x86_64)
* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/pro
System information as of Sat Mar 14 03:23:58 UTC 2026
System load: 0.0
Usage of /: 33.2% of 9.51GB
Memory usage: 35%
Swap usage: 0%
Expanded Security Maintenance for
9 updates can be applied immediat
To see these additional updates r
Enable ESM Apps to receive additi
See https://ubuntu.com/esm or run
New release '24.04.4 LTS' availab
Run 'do-release-upgrade' to upgra
Last login: Sat Mar 14 03:13:50 2
ontariotechthrowaway@scenario-vm:
Upload
Upload files from your computer
Choose Files
Phase 1 - Monolithic System.zip
Cancel
Upload Files
Last login: Sat Mar 14 03:13:50 2026 from 35.235.245.128
ontariotechthrowaway@scenario-vm:~$
Transferred 1 item
Phase 1 - Monolit...
ontariotechthrowaway@scenario-vm:~$ ls
'Phase 1 - Monolithic System' 'Phase 1 - Monolithic System.zip'
ontariotechthrowaway@scenario-vm:~$ cd 'Phase 1 - Monolithic System'
ontariotechthrowaway@scenario-vm:~/Phase 1 - Monolithic System$ ls
ngsim-scenario-extractor
ontariotechthrowaway@scenario-vm:~/Phase 1 - Monolithic System$ cd ngsim-scenario-extractor
ontariotechthrowaway@scenario-vm:~/Phase 1 - Monolithic System/ngsim-scenario-extractor$ ls
Car_Following.gif Cut-In.gif Lane_Change.gif data main.py requirements.txt scenario_output.csv src
ontariotechthrowaway@scenario-vm:~/Phase 1 - Monolithic System/ngsim-scenario-extractor$ pip3 install -r requirements.txt
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pandas in /home/ontariotechthrowaway/.local/lib/python3.10/site-packages (from -r requirements.txt (line 1)) (2.3.3)
Requirement already satisfied: matplotlib in /home/ontariotechthrowaway/.local/lib/python3.10/site-packages (from -r requirements.txt (line 2)) (3.10.8)
Requirement already satisfied: imageio in /home/ontariotechthrowaway/.local/lib/python3.10/site-packages (from -r requirements.txt (line 3)) (2.37.3)
Requirement already satisfied: numpy>=1.22.4 in /home/ontariotechthrowaway/.local/lib/python3.10/site-packages (from pandas->-r requirements.txt (line 1)) (2.2.6)
Requirement already satisfied: pytz>=2020.1 in /usr/lib/python3/dist-packages (from pandas->-r requirements.txt (line 1)) (2022.1)
Requirement already satisfied: tzdata>=2022.7 in /home/ontariotechthrowaway/.local/lib/python3.10/site-packages (from pandas->-r requirements.txt (line 1)) (2025.3)
Requirement already satisfied: python-dateutil>=2.8.2 in /home/ontariotechthrowaway/.local/lib/python3.10/site-packages (from pandas->-r requirements.txt (line 1)) (2.9.0.post0)
Requirement already satisfied: contourpy>=1.0.1 in /home/ontariotechthrowaway/.local/lib/python3.10/site-packages (from matplotlib->-r requirements.txt (line 2)) (1.3.2)
Requirement already satisfied: pyparsing>=3 in /home/ontariotechthrowaway/.local/lib/python3.10/site-packages (from matplotlib->-r requirements.txt (line 2)) (3.3.2)
Requirement already satisfied: packaging>=20.0 in /usr/lib/python3/dist-packages (from matplotlib->-r requirements.txt (line 2)) (21.3)
Requirement already satisfied: cycler>=0.10 in /home/ontariotechthrowaway/.local/lib/python3.10/site-packages (from matplotlib->-r requirements.txt (line 2)) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /home/ontariotechthrowaway/.local/lib/python3.10/site-packages (from matplotlib->-r requirements.txt (line 2)) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /home/ontariotechthrowaway/.local/lib/python3.10/site-packages (from matplotlib->-r requirements.txt (line 2)) (1.5.0)
Requirement already satisfied: pillow>=8 in /home/ontariotechthrowaway/.local/lib/python3.10/site-packages (from matplotlib->-r requirements.txt (line 2)) (12.1.1)
Requirement already satisfied: six>=1.5 in /usr/lib/python3/dist-packages (from python-dateutil>=2.8.2->pandas->-r requirements.txt (line 1)) (1.16.0)
ontariotechthrowaway@scenario-vm:~/Phase 1 - Monolithic System/ngsim-scenario-extractor$



5. Example Inputs and Outputs and Scenario Extraction

This section demonstrates how the system processes traffic trajectory data to extract driving scenarios.

The system uses sliding windows of approximately five seconds (50 frames) to analyze vehicle behavior over time. Because the windows overlap, a single driving event can appear across multiple windows.

For example:
1723–1772 Cut-In
1724–1773 Cut-In
1725–1774 Cut-In

These entries correspond to the same cut-in maneuver detected across consecutive windows.

This approach allows the system to accurately capture the duration and evolution of traffic interactions.

Example Scenario 1: Cut-in

Input

The system receives vehicle trajectory data in CSV format. Each row represents the state of a vehicle at a particular frame.

Example input data:

Vehicle_ID	Frame_ID	Lane_ID	Local_Y	Vehicle_ID	Frame_ID
15	1723	3	124.5	15	1723
24	1723	2	121.3	24	1723
24	1724	3	122.9	24	1724
24	1725	3	125.1	24	1725
24	1726	3	128.2	24	1726

Observations:

- Vehicle 24 begins in lane 2
- It moves into lane 3
- It appears ahead of vehicle 15

This maneuver corresponds to a cut-in scenario.

Output

After processing the sliding windows, the system produces an output file describing the detected scenarios.

Example output (CSV):

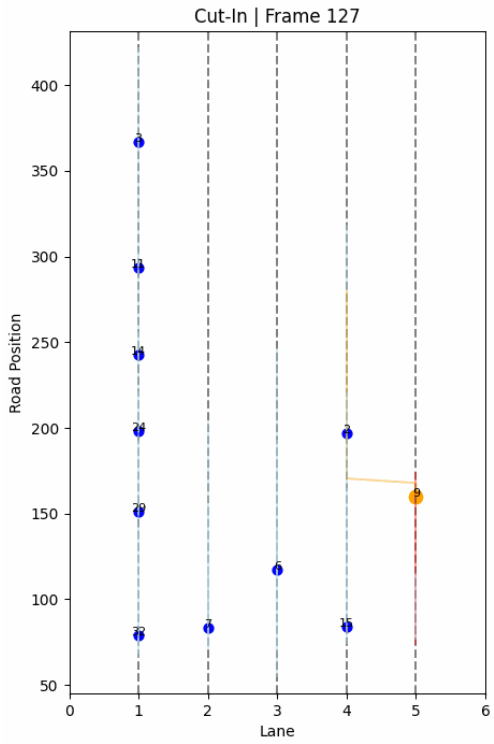
start_frame,end_frame,num_vehicles,scenario

1723,1772,57,Cut-In

1724,1773,57,Cut-In

1725,1774,57,Cut-In

The following is a visualization provided in our monolithic application. The dynamic GIF can be seen here: Cut-In.gif. The ego vehicle is in red.



Example Scenario 2: Car Following

Input

A car-following scenario occurs when two vehicles travel in the same lane with a small distance between them and similar speeds.

Example trajectory data:

Vehicle_ID	Frame_ID	Lane_ID	Local_Y	Velocity

15	200	3	120.0	22.3
18	200	3	140.5	22.1
15	201	3	122.2	22.4
18	201	3	142.6	22.0
15	202	3	124.4	22.3
18	202	3	144.7	22.2

Observations:

- Both vehicles are in lane 3
- Distance between vehicles is approximately 20 meters
- Velocity difference is very small

This behavior satisfies the car-following conditions.

Output

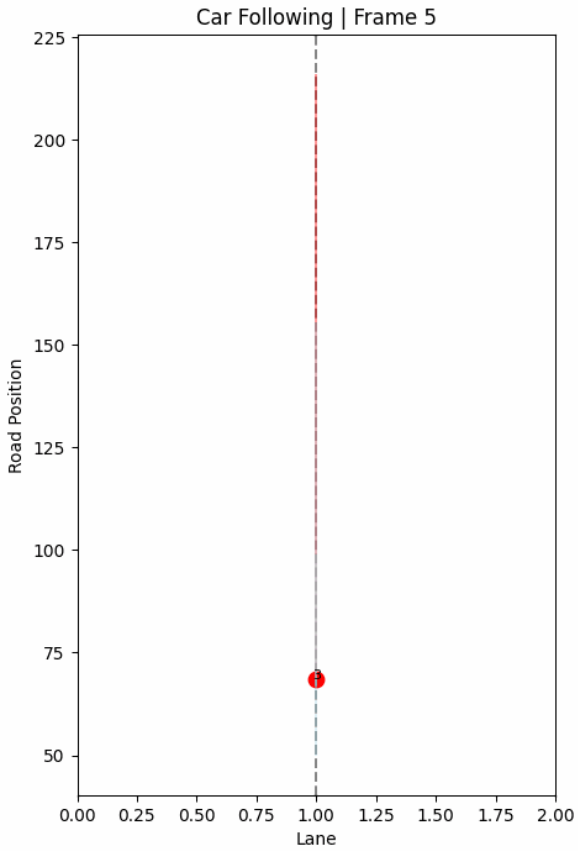
The system classifies the sliding windows containing this interaction as Car Following.

Example output:

start_frame,end_frame,num_vehicles,scenario
200,249,5,Car Following
201,250,5,Car Following
202,251,5,Car Following

This indicates that the system detected a car-following interaction during this time interval.

The following is a visualization provided in our monolithic application. The dynamic GIF can be seen here: Car_Following.gif. The ego vehicle is in red.



Example Scenario 3: Lane Change

Input

A lane change scenario occurs when a vehicle transitions from one lane to another during the observation window.

Example trajectory data:

Vehicle_ID	Frame_ID	Lane_ID	Local_Y
24	500	2	210.3
24	501	2	212.4
24	502	3	214.6
24	503	3	216.8
24	504	3	219.0

Observations:

- Vehicle 24 starts in lane 2
- The vehicle transitions to lane 3
- The lane identifier changes within the observation window

This indicates a lane change maneuver.

Output

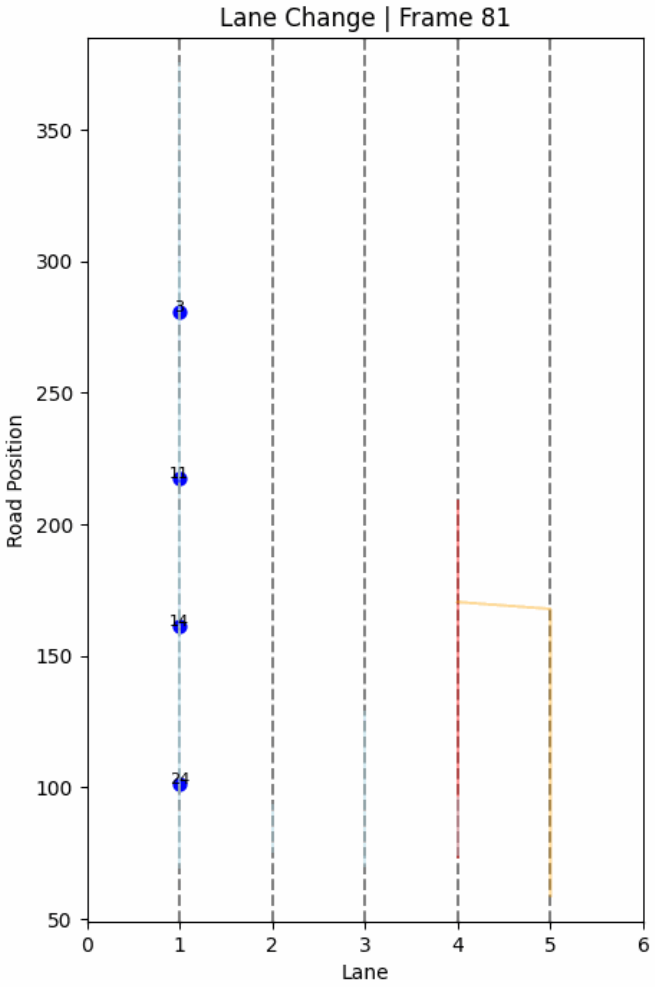
The sliding windows covering this event are classified as Lane Change.

Example output:

start_frame,end_frame,num_vehicles,scenario
500,549,7,Lane Change
501,550,7,Lane Change
502,551,7,Lane Change

Because the system uses overlapping sliding windows, the same maneuver can appear in multiple detected windows.

The following is a visualization provided in our monolithic application. The dynamic GIF can be seen here: Lane_Change.gif. The ego vehicle is in red.


Lane Change | Frame 81
Road Position
Lane
350
300
250
200
150
100
50
0
1
2
3
4
5
6